

# Quiz — 1.2.4 Types of Programming Language

---

Name: \_\_\_\_\_ Date: \_ **Mark:** \_\_\_\_ / 25

OCR H446 · Component 01 · 1.2.4

---

## Section A — Multiple choice (8 marks)

---

1. A *programming paradigm* is best described as: A. A specific programming language B. A style or approach to writing programs C. A type of CPU D. A piece of utility software Your answer: ☐
  2. The **procedural** paradigm organises a program using: A. Classes and objects only B. Sequence, selection and iteration in procedures/subroutines C. Logic facts and rules D. Spreadsheets and macros Your answer: ☐
  3. Assembly language uses **mnemonics** such as `ADD` and `STA` and is: A. A high-level, machine-independent language B. Processor-specific, with  $\approx$  one mnemonic per machine instruction C. Only usable inside a web browser D. Identical on every processor Your answer: ☐
  4. In the LMC, the instruction `BRZ xx` branches to address `xx` only if the Accumulator is: A. Greater than zero B. Equal to zero C. Less than zero D. Negative Your answer: ☐
  5. In **immediate** addressing, the operand holds: A. The actual value to be used B. The address of the value C. The address of a location holding the address of the value D. A base address plus an index Your answer: ☐
  6. **Indirect** addressing means the operand is: A. The actual value B. The address holding the value C. The address of a location that holds the address of the value D. A value added to the index register Your answer: ☐
  7. Which OOP concept means a subclass acquires the attributes and methods of a superclass? A. Encapsulation B. Inheritance C. Polymorphism D. Instantiation Your answer: ☐
  8. An **object** is best defined as: A. A template defining attributes and methods B. A specific instance of a class C. A subroutine belonging to a class D. A private variable of a class Your answer: ☐
- 

## Section B — Short answer (12 marks)

---

9. Define the OOP terms **class**, **attribute** and **method**. [3]

---

---

---

10. Explain **encapsulation** and state one benefit it provides. [2]

---

---

11. **Addressing-mode calculation.** A processor executes `LDA 6`. The relevant memory contents are:

Address 6 holds the value **9** · Address 9 holds the value **4** · The index register holds **2**.

State the value loaded into the Accumulator for each mode. **[3]**

(a) Immediate: \_\_\_ (b) **D**irect: \_ (c) **I**ndirect: \_\_\_\_

(d) Indexed (value taken from the resulting address): \_\_\_\_\_ (state which address is used)

12. Explain **two** reasons a programmer might choose to write part of a program in assembly language rather than a high-level language. **[2]**

13. Explain what **polymorphism** is and give a short example of how it could be used. **[2]**

## Section C — Exam-style question (5 marks)

14. A team is writing a program that models different types of vehicle (Car, Bus, Motorbike), all of which share common data such as `speed` and behaviour such as `move()`, but each displays itself differently. Explain how **inheritance** and **polymorphism** could be used in this object-oriented program, and give **one** benefit of designing it this way. **[5]**

## Answer key

**Section A (8 marks — 1 each)** 1. **B** — a paradigm is a style/approach to writing programs. 2. **B** — procedural code uses sequence, selection and iteration organised into procedures. 3. **B** — assembly is low-level, processor-specific,  $\approx$  one mnemonic per machine instruction. 4. **B** — BRZ branches only when the Accumulator equals zero. 5. **A** — immediate: the operand is the actual value. 6. **C** — indirect: operand points to a location that holds the address of the value (pointer). 7. **B** — inheritance: a subclass acquires the superclass's attributes/methods. 8. **B** — an object is a specific instance of a class.

### Section B (12 marks)

9. **[3]** Class = a template/blueprint defining attributes and methods (1); attribute = a variable of a class/object holding its data/state (1); method = a subroutine of a class defining its behaviour (1).

10. **[2]** Encapsulation = bundling data and the methods that act on it together, with the data hidden (private attributes accessed only via methods) (1). Benefit: protects data from invalid/unauthorised changes / makes code easier to maintain because internals can change without affecting outside code (1).

11. **[3] Addressing-mode calculation** — for `LDA 6` (operand = 6): - (a) **Immediate = 6** — the operand is the value itself (1). - (b) **Direct = 9** — load the value held at address 6, which is 9 (1). - (c) **Indirect = 4** — address 6 holds 9, so go to address 9, whose value is 4 (1). - (d) **Indexed = 4, using address 6 + 2 = 8** — base address 6 + index 2 = address 8; value taken from address 8. (Accept the candidate's correct value if they are given/assume address 8's contents; the key skill is computing the effective address  $6 + 2 = 8$ .) (Not separately marked — included to reinforce method; award the 3 marks across (a)–(c). If a centre marks (d), accept "address 8" as the effective address.)

*Marking note: 3 marks awarded for (a) 6, (b) 9, (c) 4. Part (d) checks the effective-address method ( $6 + 2 = 8$ ); reward in feedback.*

12. **[2]** Any two (1 each): faster execution / smaller program; finer/direct control of the hardware (e.g. registers, peripherals); needed for device drivers or embedded systems with limited memory; real-time code where speed is critical.
13. **[2]** Polymorphism = the same method name behaves differently depending on the object it is called on (typically via overriding) (1). Example: a `draw()` or `area()` method gives different results for a Circle vs a Square object, or `speak()` returns "Woof" for a Dog and "Meow" for a Cat (1).

**Section C (5 marks)** 14. **[5]** Up to 5 from: define a **superclass** `Vehicle` holding the shared attributes (e.g. `speed`) and shared method `move()` (1); `Car`, `Bus` and `Motorbike` are **subclasses that inherit** these attributes/methods rather than redefining them (1); each subclass **overrides** a method (e.g. `display()`) so it behaves differently per object — this is **polymorphism** (1); the program can hold/treat all vehicles uniformly (e.g. in one list) yet each responds correctly when its method is called (1). Benefit (1): reduces duplicated code / DRY, so it is easier to maintain and extend (adding a new vehicle type only requires a new subclass).

**Total: 8 + 12 + 5 = 25**